

Credit Card Fraud Detection System

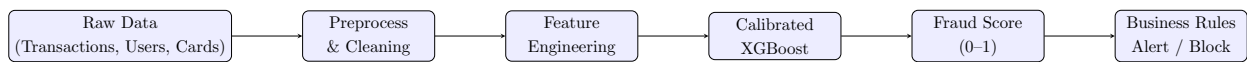
GitHub Project Summary

Author: Rushikesh Shirwalkar

Overview

This project implements a credit card fraud detection engine trained on more than 13 million transactions. The system assigns a fraud probability score to each transaction and allows operational teams to configure business thresholds for alerts. The final solution uses feature-engineered, calibrated XGBoost, with Random Forest as a baseline.

System Architecture



This architecture reflects a production-style risk scoring pipeline: raw data flows through preprocessing and feature engineering into a calibrated model whose outputs feed downstream fraud decisioning.

Dataset

- **Transactions:** 13M+ card transactions with timestamp, amount, merchant, city/state, MCC and chip usage.
- **Users:** demographic and financial attributes (age, income, credit score, debt, number of cards).
- **Cards:** card brand, type, credit limit, chip, dark-web flag, number of cards issued.
- **MCC Codes:** merchant category taxonomy.
- **Fraud Labels:** binary labels (Yes/No) for a subset of transactions.

Fraud accounts for less than 0.03% of all transactions, making this a highly imbalanced classification problem.

Feature Engineering Summary

High-impact features were engineered on top of raw fields:

- **User behaviour:** average transaction amount, standard deviation, deviation from user's typical spend, spend ratios, daily transaction counts.
- **Merchant risk:** historical fraud rate, average ticket size, deviation of current amount from merchant norm.
- **Temporal patterns:** hour of day, weekday vs weekend, late-night indicator, bursts of activity within a day.

These behavioural and contextual features significantly improved model separation compared to using only raw transaction attributes.

Performance Metrics

On a balanced evaluation set (fraud vs non-fraud sampled):

- **Precision:** 0.88
- **Recall:** 0.92
- **F1-Score:** 0.90
- **ROC-AUC:** 0.9907

These metrics show that the calibrated XGBoost model captures the fraud signal effectively when evaluated on balanced data.

Random Forest was used as a baseline and achieved ROC-AUC around 0.71, confirming that gradient boosting with rich features is better suited for this problem.

Threshold Strategy

Two thresholds are important in this system:

- **Default threshold (0.50)** – useful for demonstrating model quality with standard ML metrics (precision, recall, F1, ROC-AUC).
- **Business threshold (e.g. 0.14)** – chosen from a precision-recall curve; increases recall to catch more fraud, while accepting some extra false positives. This reflects real fraud operations where “missing fraud” is more costly than reviewing a few extra alerts.

In practice, different products or regions can use different thresholds based on risk appetite.

How to Run

1. Install Requirements

```
pip install -r requirements.txt
```

2. Load the Model and Threshold

```
import joblib
```

```
model = joblib.load("models/fraud_detection_pipeline.pkl")
```

```
threshold = joblib.load("models/fraud_threshold.pkl")
```

```
\end{verbatim}
```

```
\subsection*{3. Score New Transactions}
```

```
\begin{verbatim}
```

```
proba = model.predict_proba(X_new)[: , 1]    # fraud probabilities
```

```
pred  = (proba > threshold).astype(int)      # 1 = fraud, 0 = legitimate
```

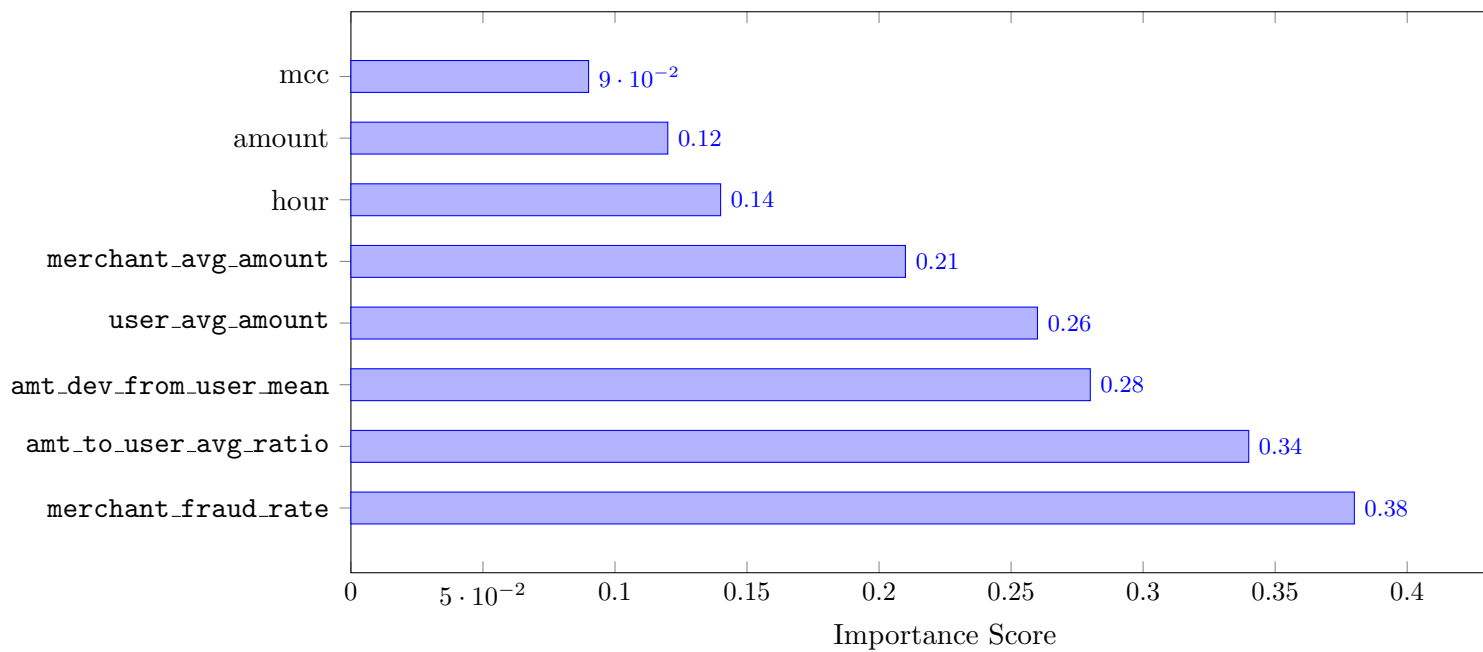
4. Integration

Use `pred == 1` to:

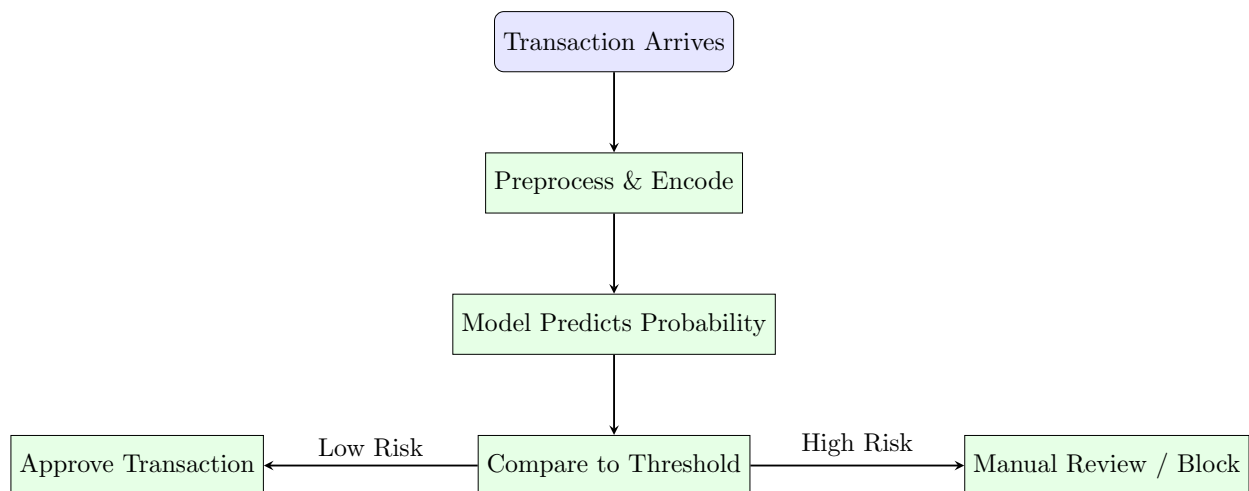
- challenge the transaction (e.g. OTP / step-up auth),
- send the case to a manual review queue,
- block the payment if risk is deemed too high.

Feature Importance (XGBoost)

The plot below shows an illustrative ranking of key features in the trained XGBoost model. Behavioural and merchant-risk variables dominate the signal.



Fraud Alert Flow



Key Takeaways

- Extreme class imbalance requires careful modeling and threshold design; accuracy alone is not meaningful.
- Behaviour- and merchant-based features provide much stronger signal than raw transaction attributes.
- Calibrated XGBoost delivers high-quality fraud probabilities suitable for risk scoring.
- RandomForest is a useful baseline but underperforms compared to the boosted model.
- Business thresholds connect model outputs to real-world costs and operational capacity.